

สัปดาห์ที่ 1

ทบทวนพื้นฐาน JavaScript

Condition

Loop

Array

Object

Function

JavaScript Module

JavaScript Date

Regular Expression

Exception Handling

01

VARIABLE | CONDITION | LOOP

ตัวแปร | เงื่อนไข | ลูป

ตัวแปรและชนิดข้อมูล (Variables & Data Types)

การประกาศตัวแปร

- `let` — ค่าที่เปลี่ยนแปลงได้ ขอบเขตแบบ block
- `const` — ค่าคงที่ ห้ามกำหนดค่าใหม่
- `var` — รูปแบบเก่า (ควรหลีกเลี่ยง)

ชนิดข้อมูลพื้นฐาน (Primitive Types)

String, Number, Boolean, null, undefined

ตรวจสอบชนิดข้อมูลด้วย `typeof`

```
let score = 85;
const studentName = "Nan";
var isPassed = true;    // ควรหลีกเลี่ยงการใช้ var

let price;              // undefined
let empty = null;       // ตั้งใจให้เป็นค่าว่าง

console.log(typeof score) ;    // "number"
console.log(typeof studentName) ; // "string"
console.log(typeof isPassed) ;  // "boolean"
```

เงื่อนไข และการวนซ้ำ (Conditionals & Loops)

เงื่อนไข (Conditionals)

- if / else if / else
- switch — เมื่อมีหลายกรณีให้เทียบ

การวนซ้ำ (Loops)

- for — เมื่อรู้จำนวนรอบที่แน่นอน
- while — เมื่อวนซ้ำตามเงื่อนไข

```
let score = 85;

if (score >= 80) {
  console.log("A");
} else if (score >= 70) {
  console.log("B");
} else {
  console.log("need improvement");
}

for (let i = 1; i <= 5; i++) {
  console.log("รอบที่ " + i);
}
```

Switch สามารถใช้แทน if/else ที่มีหลายเงื่อนไข

- เปรียบเทียบค่าตัวแปรตัวเดียวกับหลายกรณี (case) ที่กำหนดไว้
- ใช้ strict equality (===) ในการเทียบค่าเสมอ
- break หยุดการทำงานเมื่อเจอ case ที่ตรง ไม่เช่นนั้นจะ 'ไหล' ต่อไป case ถัดไป
- default ทำงานเมื่อไม่มี case ไหนตรงเลย (จะใส่หรือไม่ใส่ก็ได้)

```
const day = 3;
let dayName;

switch (day) {
  case 1: dayName = "จันทร์"; break;
  case 2: dayName = "อังคาร"; break;
  case 3: dayName = "พุธ"; break;
  default: dayName = "ไม่ทราบวัน";
}

console.log(dayName); // "พุธ"
```

เลือกใช้ Switch หรือ if/else เมื่อไหร่ดี

ใช้ Switch เมื่อ...

- เียบค่าตัวแปรเดียวกับค่าคงที่หลายค่า (enum-like)
- จำนวน case มีมากกว่า 3-4 กรณีขึ้นไป
- ต้องการให้โค้ดอ่านเป็นรายการ ชัดเจนกว่า if ซ้อนกันยาว ๆ

```
switch (status) {  
  case "pending": ...  
  case "shipped": ...  
  case "delivered": ...  
}
```

ใช้ if/else เมื่อ...

- เงื่อนไขเป็นช่วงตัวเลข (range) เช่น คะแนน ≥ 80
- ต้องเทียบหลายตัวแปรพร้อมกัน (&&, ||)
- เงื่อนไขซับซ้อน ไม่ใช่แค่ 'ค่าเท่ากับ' ตรง ๆ

```
if (score  $\geq$  80) {  
  grade = "A";  
} else if (score  $\geq$  70) {  
  grade = "B";  
}
```

ตัวอย่าง: badge สถานะคำสั่งซื้อ

หน้าติดตามคำสั่งซื้อต้องแสดง label และสีต่างกันตามสถานะที่ได้จาก API — Switch ช่วยจัดการค่าคงที่หลายค่าแบบนี้ได้อ่านง่ายกว่า if/else ซ้อนกันยาว ๆ

```
function getOrderStatusLabel(status) {  
  switch (status) {  
    case "pending": return "รอดำเนินการ";  
    case "shipped": return "จัดส่งแล้ว";  
    case "delivered": return "ส่งถึงแล้ว";  
    case "cancelled": return "ยกเลิกคำสั่งซื้อ";  
    default: return "ไม่ทราบสถานะ";  
  }  
}  
  
getOrderStatusLabel("shipped"); // "จัดส่งแล้ว"  
getOrderStatusLabel("unknown"); // "ไม่ทราบสถานะ" (จาก default)
```

Loop

ชนิด Loop	ใช้เมื่อ...	syntax ตัวอย่าง
for	รู้จำนวนรอบที่แน่นอน	<code>for (let i=0; i<5; i++) {}</code>
while	ไม่รู้จำนวนรอบ แต่รู้เงื่อนไขจบ	<code>while (cond) {}</code>
do...while	ต้องทำอย่างน้อย 1 ครั้งก่อนเช็คเงื่อนไข	<code>do {} while (cond);</code>
for...of	วนอ่าน 'ค่า' ใน Array/String/Map/Set	<code>for (const v of arr) {}</code>
for...in	วนอ่าน 'key' ใน Object	<code>for (const k in obj) {}</code>

```
const products = ["เสื่อยี่ด", "หมวก", "กระเป๋า"];
for (const product of products) {
  console.log(`- ${product}`);
}
// ผลลัพธ์: - เสื่อยี่ด / - หมวก / - กระเป๋า
```


Loop ซ้อน Loop: ตารางเปรียบเทียบ 2 มิติ

เมื่อข้อมูลมี 2 มิติ (เช่น หมวดหมู่ x หมวดหมู่ย่อย) loop วงนอกควบคุมแกนหนึ่ง วงในควบคุมอีกแกนหนึ่ง ใช้จริงตอนเรนเดอร์ตาราง filter หรือปฏิทิน

```
const categories = ["เสื้อผ้า", "รองเท้า"];
const sizes = ["S", "M", "L"];

for (const category of categories) {
  for (const size of sizes) {
    console.log(`${category} - ไชล์ ${size}`);
  }
}
// เสื้อผ้า - ไชล์ S
// เสื้อผ้า - ไชล์ M
// เสื้อผ้า - ไชล์ L
// รองเท้า - ไชล์ S
// รองเท้า - ไชล์ M
// รองเท้า - ไชล์ L
```

for loop: break/continue

- ส่วนที่ 1 — initialization: กำหนดค่าเริ่มต้น (ทำครั้งเดียว)
- ส่วนที่ 2 — condition: เช็คก่อนวนทุกรอบ ถ้า false จะหยุดทันที
- ส่วนที่ 3 — increment: ทำงานหลังจบแต่ละรอบ
- break — ออกจาก loop ทันที ไม่วนต่อเลย
- continue — ข้ามรอบปัจจุบัน ไปเริ่มรอบถัดไปทันที

```
for (let i = 1; i <= 10; i++) {  
  if (i === 3) continue; // ข้าม 3  
  if (i === 7) break;    // หยุดที่ 7  
  console.log(i);  
}  
  
// ผลลัพธ์: 1, 2, 4, 5, 6  
  
// วน array ด้วย index เมื่อจำเป็นต้องรู้ตำแหน่ง  
const items = ["A", "B", "C"];  
for (let i = 0; i < items.length; i++) {  
  console.log(i, items[i]);  
}
```

while vs. do...while

while

เช็คเงื่อนไขก่อนทำงาน — ถ้าเท็จตั้งแต่แรก จะไม่ทำงานเลยสักครั้ง

```
let count = 0;
while (count < 3) {
  console.log(count);
  count++;
}
// พิมพ์ 0, 1, 2
```

do...while

ทำงานก่อนอย่างน้อย 1 ครั้งเสมอ แล้วค่อยเช็คเงื่อนไข

```
let count = 10;
do {
  console.log(count);
  count++;
} while (count < 3);
// พิมพ์ 10 (แม้เงื่อนไขเท็จตั้งแต่แรก)
```

ระวังสับสน: for...of กับ for...in

for...of

วนอ่าน 'ค่า' — ใช้กับ Array, String, Map, Set

```
const fruits = ["apple", "kiwi"];
for (const fruit of fruits) {
  console.log(fruit);
}
// "apple", "kiwi" (ค่าจริง)
```

for...in

วนอ่าน 'คีย์ (key)' — ใช้กับ Object เท่านั้น

```
const car = { brand: "Toyota", year: 2023 };
for (const key in car) {
  console.log(key, car[key]);
}
// "brand Toyota", "year 2023"
```

⚠️ ข้อควรระวัง: อย่าใช้ for...in กับ Array เพราะจะได้ 'index' เป็น string ("0","1",...) แทนค่าจริง ให้ใช้ for...of กับ Array เสมอ

02

FUNCTION

ฟังก์ชัน

Function — 3 รูปแบบการเขียน

// 1) Function Declaration

```
function add(a, b) {  
  return a + b;  
}
```

// 2) Function Expression

```
const add2 = function (a, b) {  
  return a + b;  
}
```

// 3) Arrow Function

```
const add3 = (a, b) => a + b;
```

จุดสังเกต

- ทั้ง 3 รูปแบบทำงานให้ผลลัพธ์เดียวกัน
- Function Declaration ถูกเรียกใช้ก่อนประกาศได้ (hoisting)
- Arrow Function นิยมใช้ในโค้ดสมัยใหม่ สั้นและกระชับ

ทำไมต้องรู้ทั้ง 3 แบบ?

เพราะโค้ดจริงในอุตสาหกรรม (และโค้ดที่ AI generate ให้) ใช้ปะปนกันทั้ง 3 แบบ ผู้เรียนต้องอ่านและเข้าใจได้ทุกรูปแบบ

Function — กลุ่มคำสั่งที่นำกลับมาใช้ซ้ำ

ฟังก์ชัน คือ กลุ่มคำสั่งที่ตั้งชื่อไว้ ทำงานเมื่อถูกเรียกใช้ (function call) เช่น ตอนกดปุ่ม login หรือกด submit φόρμ รับค่าเข้า (parameter) และคืนค่าออก (return)

Function Declaration

```
function handleLoginClick(u) {  
  return `ยินดีต้อนรับ ${u}`; }  
}
```

Function Expression

```
const isValidEmail = function(e) {  
  return e.includes('@');  
}
```

Arrow Function

```
const isValidEmail = (e) => e.includes('@');
```



```
// higher-order function: รับฟังก์ชันเข้ามาเป็น argument  
function calculateShippingFee(total, shippingFn) {  
  return shippingFn(total);  
}  
  
const standardShipping =  
(total) => (total > 1000 ? 0 : 50);  
  
calculateShippingFee(1200, standardShipping);  
// 0 (ซื้อครบ 1000 ส่งฟรี)
```

พารามิเตอร์ ค่าที่ส่งกลับ และขอบเขตตัวแปร

Parameter & Return

- Parameter คือ ตัวรับค่าที่ส่งเข้าไปในฟังก์ชัน
- return ใช้ส่งค่าผลลัพธ์กลับออกจากฟังก์ชัน
- ฟังก์ชันที่ไม่มี return จะคืนค่า undefined

ขอบเขตตัวแปร (Scope)

- Local Scope — ตัวแปรที่ประกาศในฟังก์ชัน ใช้ได้เฉพาะภายใน
- Global Scope — ตัวแปรที่ประกาศนอกฟังก์ชัน ใช้ได้ทุกที่

```
let discount = 0.1; // global scope
```

```
function getFinalPrice(price) {  
    let tax = price * 0.07; // local scope  
    return price + tax - (price * discount);  
}
```

```
console.log(getFinalPrice(100));
```

```
console.log(tax); // Error! tax ไม่รู้จักนอกฟังก์ชัน
```


03

ARRAY

อาร์เรย์

Array — เก็บชุดข้อมูลแบบลำดับ

Array คืออะไร

- ชุดข้อมูลที่เรียงลำดับ เข้าถึงด้วย index (เริ่มที่ 0)
- ตรวจสอบจำนวนสมาชิกด้วย .length

เมธอดพื้นฐานที่ต้องรู้

- push() / pop() — เพิ่ม/ลบท้ายอาร์เรย์
- forEach() — วนซ้ำทำงานกับทุกสมาชิก
- map() / filter() — แปลง/กรองข้อมูล

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits[0]); // "apple"
console.log(fruits.length); // 3

fruits.push("grape");

fruits.forEach((f) => console.log(f));

const upper = fruits.map((f) =>
  f.toUpperCase());
```

Array Methods

map()

ดึงชื่อสินค้าไปแสดงใน dropdown

```
products.map  
(p => p.name)
```

filter()

กรองเฉพาะสินค้าที่ยังมีสต็อก

```
products.filter  
(p => p.inStock)
```

reduce()

รวมยอดราคาตะกร้าตอน checkout

```
cart.reduce((t,i)=>  
t+i.price,0)
```

find()

หาสินค้าชิ้นแรกที่มี id ที่เลือก

```
products.find  
(p => p.id === 3)
```

04

OBJECT

ออปเจกต์

Object — เก็บข้อมูลแบบ key-value

Object คืออะไร

- ชุดข้อมูลแบบ key-value ใช้เก็บคุณสมบัติของสิ่งหนึ่ง ๆ
- เข้าถึงค่าได้ด้วย dot notation หรือ bracket notation

Property & Method

- Property — ข้อมูล/คุณสมบัติของอ็อบเจกต์
- Method — ฟังก์ชันที่อยู่ภายในอ็อบเจกต์
- this — อ้างอิงถึงตัวอ็อบเจกต์นั่นเอง (เบื้องต้น)

```
const student = {  
  name: "Nan",  
  score: 85,  
  greet: function () {  
    return "Hi, I am " + this.name;  
  },  
};  
  
console.log(student.name);      // dot  
console.log(student["score"]);  // bracket  
console.log(student.greet());
```

Object

- เข้าถึงค่าด้วย dot: userProfile.email
- หรือ bracket: userProfile["email"]
- แก้ไข/เพิ่ม property ได้ทันที: userProfile.theme = "dark"

```
// ข้อมูลโปรไฟล์ผู้ใช้จากหน้าตั้งค่าบัญชี
const userProfile = {
  name: "เอ",
  email: "a@mail.com",
  theme: "light"
};

console.log(userProfile.email);

// กดปุ่ม Dark Mode → แก้ไขค่า property เดียว
userProfile.theme = "dark";
```

05

SPREAD| DESTRUCTURING

กระจายและแยกค่าออกจาก Array และ Object

Spread (...) กระจายสมาชิก Array ออกมา

- คัดลอก array แบบไม่อ้างอิงตัวเดิม (copy by value ระดับบนสุด)
- รวม 2 array เข้าด้วยกันแบบง่าย ๆ ไม่ต้องใช้ concat()
- กระจายค่าเป็น argument ของฟังก์ชัน เช่น Math.max(...arr)
- แทรกสมาชิกใหม่เข้าไปได้ทุกตำแหน่งที่ต้องการ

```
const a = [1, 2];
const b = [...a, 3, 4];
console.log(b); // [1, 2, 3, 4]

// รวม 2 array
const merged = [...a, ...b];

// กระจายเป็น argument
const nums = [3, 7, 2];
console.log(Math.max(...nums)); // 7
```


Spread กับ Object: คัดลอกและรวมค่า

- คัดลอก object ระดับบนสุดได้ในบรรทัดเดียว { ...obj }
- รวมหลาย object เข้าด้วยกันเป็นก้อนเดียว
- ลำดับสำคัญมาก: key ที่ซ้ำกัน ค่าจาก object 'ตัวหลัง' จะทับตัวก่อนหน้าเสมอ
- ใช้บ่อยเวลาอัปเดต state บางส่วนโดยไม่แก้ค่าตัวเดิม (immutability)

```
const defaults = { theme: "light", lang: "th" };
const userPrefs = { theme: "dark" };

const settings = { ...defaults, ...userPrefs };
console.log(settings);
// { theme: "dark", lang: "th" }
// userPrefs มาทีหลัง จึงทับค่า theme เดิม

// เพิ่ม/แก้ property เดียว โดยไม่แตะของเดิม
const updated = { ...settings, lang: "en" };
```

Spread และ Rest หน้าตาเหมือนกัน (...) แต่ทำหน้าที่ตรงข้ามกัน

Spread — 'กระจาย' ค่าออก

ใช้ตอนเรียกใช้ (เช่น เรียกฟังก์ชัน หรือสร้าง array/object ใหม่)

```
const nums = [1, 2, 3];  
sum(...nums); // กระจาย 3 ค่าเข้า argument
```

Rest — 'รวบรวม' ค่าเข้า

ใช้ตอนประกาศฟังก์ชัน เพื่อรับ argument ไม่จำกัดจำนวนเป็น array เดียว

```
function sum(...numbers) {  
  return numbers.reduce((a,b)=>a+b, 0);  
}  
sum(1, 2, 3); // 6
```

ตัวอย่าง: อัปเดต state แบบไม่แก้ของเดิม



```
// เพิ่มสินค้าเข้าตะกร้า โดยไม่แก้ array เดิม
const cart = ["เสื้อยืด", "หมวก"];
const newCart = [...cart, "รองเท้า"];
// cart เดิมไม่ถูกกระทบ, newCart คือก้อนใหม่ที่มี 3 ชิ้น

// แก้ค่า property เดียวใน user object โดยไม่แก้ตัวเดิม
const user = { name: "เอ", theme: "light" };
const updatedUser = { ...user, theme: "dark" };
// user เดิมยังเป็น theme: "light" อยู่เหมือนเดิม
```

Array Destructuring: skip, default, swap

- แยกค่าตามลำดับตำแหน่ง ไม่ใช่ตามชื่อ
- ข้ามตำแหน่งที่ไม่ต้องการด้วยการเว้นช่องว่างไว้ (,)
- ตั้งค่า default ได้ ถ้าตำแหน่งนั้นเป็น undefined
- สลับค่าตัวแปร 2 ตัวได้ในบรรทัดเดียว ไม่ต้องใช้ temp

```
const [a, b, c] = [10, 20, 30];

// ข้ามตัวที่ 2 (20)
const [x, , z] = [1, 2, 3];
console.log(z); // 3

// ตั้งค่า default ถ้าตำแหน่งนั้นว่าง
const [p = 0, q = 0] = [5];
console.log(p, q); // 5 0

// สลับค่าตัวแปร (swap) ในบรรทัดเดียว
let m = 1, n = 2;
[m, n] = [n, m]; // m=2, n=1
```

Object Destructuring: rename, default, nested

- แยกค่าตาม 'ชื่อ key' ไม่ใช่ตามลำดับเหมือน array
- ตั้งชื่อตัวแปรใหม่ได้ด้วย key: newName
- ตั้งค่า default ได้เมื่อ key นั้นไม่มีอยู่จริงใน object
- แยก object ที่ซ้อนกันหลายชั้นได้ในนิพจน์เดียว

```
const user = { name: "เอ", city: "กทม." };
const { name, city } = user;

// ตั้งชื่อตัวแปรใหม่ (rename)
const { name: fullName } = user;

// ตั้งค่า default เมื่อ key ไม่มีอยู่จริง
const { country = "ไทย" } = user;

// nested destructuring
const order = { id: 1, customer: { name: "บี" } };
const { customer: { name: buyerName } } = order;
```

Destructure ตรง Parameter ของฟังก์ชัน

แทนที่จะรับ object/array เต็มก้อนแล้วค่อยแกะทีหลัง เราสามารถ destruct 'ตรง parameter' ได้เลย ทำให้โค้ดในฟังก์ชันอ่านง่ายขึ้นทันที

Object parameter

```
function ProductCard({ name, price }) {  
  return `${name} - ${price} บาท`;  
}  
  
ProductCard({ name: "หมวก", price: 150 });
```

Object parameter + default

```
function greetUser({ name, role = "guest" }) {  
  console.log(`${name} (${role})`);  
}  
  
greetUser({ name: "เอ" }); // เอ (guest)
```

Destructuring — แยกค่าออกมาใช้แบบรวดเร็ว

แทนที่จะเข้าถึงค่าทีละตัว เราสามารถ 'แยก' ค่าจาก API response หรือ props ออกมาเก็บในตัวแปรได้ในบรรทัดเดียว ช่วยให้โค้ดอ่านง่ายและสั้นลง

Array Destructuring (API response)

```
const response = [200, "สั่งซื้อสำเร็จ"];
const [statusCode, message] = response;

console.log(statusCode); // 200
console.log(message);    // ข้อความ
```

Object Destructuring (props สินค้า)

```
const product = { name: "หมวก", price: 150 };
function ProductCard({ name, price }) {
  return `${name} - ${price} บาท`;
}

ProductCard(product); // ตรง parameter เลย
```

ตัวอย่าง: แยกข้อมูลจาก API response

API มักห่อข้อมูลและบางฟิลด์อาจไม่มีค่า ต้องผสาน nested destructuring กับ default value เพื่อป้องกัน error ตอนแสดงผล

```
const apiResponse = {
  data: {
    user: { name: "มิว", avatar: null },
  },
};

const {
  data: {
    user: { name, avatar = "default.png" },
  },
} = apiResponse;

console.log(name, avatar); // "มิว" "default.png"
```


06

JAVASCRIPT DATE

จัดการวันที่และเวลาในหน้าเว็บ

สร้าง Date ได้หลายวิธี

- `new Date()` — urrent เวลาปัจจุบัน ณ ตอนนั้น
- `new Date(year, month, day)` — กำหนดวันที่เอง (เดือนเริ่มจาก 0!)
- `new Date("2026-07-30")` — แปลงจาก string รูปแบบ ISO
- `Date.now()` — คืนค่าเป็นตัวเลข timestamp (มิลลิวินาที) ไม่ใช่ object

```
const now = new Date();

// ⚠ เดือนนับจาก 0 (0=มกราคม, 6=กรกฎาคม)
const newYear = new Date(2026, 0, 1);

// จากค่า string รูปแบบ ISO 8601
const orderDate = new Date("2026-07-30");

// timestamp เป็นตัวเลข (มิลลิวินาที)
const ts = Date.now();
```

เมธอดดึงส่วนประกอบของวันที่ (getters)

เมธอด	ความหมาย	ตัวอย่างผลลัพธ์
<code>getFullYear()</code>	ปี ค.ศ. 4 หลัก	2026
<code>getMonth()</code>	เดือน (0-11  เริ่มจาก 0)	6 → กรกฎาคม
<code>getDate()</code>	วันที่ในเดือน (1-31)	30
<code>getDay()</code>	วันในสัปดาห์ (0=อาทิตย์...6=เสาร์)	4 → พฤหัสบดี
<code>getHours()</code> / <code>getMinutes()</code>	ชั่วโมง / นาที ปัจจุบัน	14 / 32

แปลง Date เป็นข้อความแสดงผลบนหน้าเว็บ

- `toLocaleString('th-TH')` — แสดงวันที่เวลาตามรูปแบบไทย
- `toLocaleDateString('th-TH')` — แสดงวันที่ตามรูปแบบไทย
- `toLocaleTimeString('th-TH')` — แสดงเวลาตามรูปแบบไทย
- ใส่ options กำหนดรูปแบบเอง เช่น วันแบบเต็ม (long) หรือแบบย่อ (short)
- `toISOString()` — รูปแบบมาตรฐานวันที่เวลาสากลสำหรับส่งข้อมูลไป API

```
const orderDate = new Date("2026-07-30T14:32:00");

orderDate.toLocaleString("th-TH");
//30/7/2569 14:32:00
orderDate.toLocaleDateString("th-TH");
//30/7/2569 (ปี พ.ศ.)
orderDate.toLocaleTimeString("th-TH");
//14:32:00

orderDate.toLocaleDateString("th-TH", {
  day: "numeric", month: "long", year: "numeric"
}); //30 กรกฎาคม 2569

// ส่งค่าไป API ด้วยรูปแบบมาตรฐาน
orderDate.toISOString(); // "2026-07-30T07:32:00.000Z"
```

บวก/ลบวันที่ และหาผลต่างระหว่างวัน

- Date ไม่มีเมธอด addDays() ตรง ๆ ต้องใช้ setDate() ร่วมกับ getDate()
- การลบ Date สองตัว จะได้ผลต่างเป็น 'มิลลิวินาที' เสมอ
- แปลงมิลลิวินาทีเป็นวันด้วยการหาร $1000 \times 60 \times 60 \times 24$

```
// บวกวันที่ไป 7 วัน (เช่น กำหนดส่งของ)
const orderDate = new Date();
const deliveryDate = new Date(orderDate);
deliveryDate.setDate(orderDate.getDate() + 7);

// หาผลต่างระหว่างวันนี้กับวันหมดโปรโมชั่น
const promoEnd = new Date("2026-08-05");
// 2026-08-05T00:00:00.000Z
const currentDate = new Date();
// 2026-07-30T10:07:34.569Z
const diffMs = promoEnd - currentDate;
const diffDays = Math.ceil(diffMs / (1000 * 60 * 60 * 24));

console.log(`เหลืออีก ${diffDays} วัน`);

// เหลืออีก 6 วัน
```

ตัวอย่าง: นับถอยหลังระยะเวลาโปรโมชั่นหมดเขต

หน้าโปรโมชั่นต้องอัปเดตเวลาถอยหลังแบบเรียลไทม์ — ใช้ setInterval ร่วมกับการคำนวณผลต่างวันที่ที่เพิ่งเรียนไป

```
// main.js
function updateCountdown(endDate) {
  const now = new Date();
  const diffMs = endDate - now;

  if (diffMs <= 0) return "หมดเขตโปรโมชั่นแล้ว";

  const hours = Math.floor(diffMs / (1000*60*60));
  const minutes = Math.floor((diffMs / (1000*60)) % 60);

  return `เหลืออีก ${hours} ชม. ${minutes} นาที`;
}

setInterval(() => {
  document.getElementById('promoRemainTime').textContent = updateCountdown(
    new Date('2026-09-30T18:00:00');
  )
}, 60000); // อัปเดตทุก 1 นาที
```

```
<!doctype html>
<html lang="en">
  <head>
    <title>Document</title>
  </head>
  <body>
    Our promotion remaing time:
    <label id="promoRemainTime">
      Waiting...
    </label>
    <script type="module"
      src="./main.js"></script>
  </body>
</html>
```

07

REGULAR EXPRESSION

ค้นหาและตรวจสอบรูปแบบข้อความ

Regular Expression คืออะไร

- รูปแบบ (pattern) สำหรับค้นหา/ตรวจสอบ/แทนที่ข้อความ
- เขียนด้วยเครื่องหมาย / / เช่น /abc/ หรือสร้างจาก new RegExp()
- ใส่ flag ต่อท้ายเพื่อปรับพฤติกรรม: g (หาทุกจุดที่ตรง), i (ไม่สนตัวพิมพ์เล็ก-ใหญ่)
- ใช้คู่กับเมธอดของ string/regex เพื่อตรวจสอบหรือดึงข้อมูล

```
// ตรวจสอบว่า string มีคำว่า "error" หรือไม่ (ไม่สนตัวพิมพ์)
const pattern = /error/i;

console.log(pattern.test("Error: not found"));
// true

console.log(pattern.test("everything is fine"));
// false
```


สัญลักษณ์ที่ใช้บ่อยใน Pattern

สัญลักษณ์	ความหมาย	ตัวอย่าง
\d	ตัวเลข 0-9	^d+/ → "123"
\w	ตัวอักษร/ตัวเลข/underscore	^w+/ → "user_01"
\s	ช่องว่าง (space, tab)	^s/ → เว้นวรรค
^	จุดเริ่มต้น	^Hi/ → ต้องขึ้นต้นด้วย Hi
\$	จุดสิ้นสุดของข้อความ	/\$Hi/ → ต้องลงท้ายด้วย Hi
* + ? {n,m}	จำนวนซ้ำ: 0+, 1+, 0-1, n ถึง m ครั้ง	^d{3}/ → เลข 3 หลักพอดี

test(), match(), replace(), split()

test()

ตรวจว่าตรง pattern หรือไม่
(true/false)

```
regex.test(str)
```

match()

ดึงส่วนที่ตรง pattern ออกมา

```
str.match(regex)
```

replace()

แทนที่ส่วนที่ตรง pattern

```
str.replace(regex, newStr)
```

split()

แบ่ง string ตาม pattern

```
str.split(regex)
```

```
"เบอร์ 081-234-5678".replace(/-/g, ""); // "เบอร์ 0812345678"
```

Pattern ตรวจสอบข้อมูลในแบบฟอร์มที่ใช้บ่อย

```
// ตรวจสอบรูปแบบอีเมลเบื้องต้น
const emailPattern = /^[\\w.-]+@[\\w-]+\\. [a-z]{2,}$ /i;
emailPattern.test("a@mail.com"); // true

// ตรวจสอบเบอร์โทรไทย 10 หลัก ขึ้นต้นด้วย 0
const phonePattern = /^0\\d{9}$ /;
phonePattern.test("0812345678"); // true

// ตรวจสอบรหัสผ่าน: ต้องมีตัวเลข, ตัวพิมพ์ใหญ่, ยาว 8+
const hasNumber = /\\d /;
const hasUpper = /[A-Z] /;
hasNumber.test(password) && hasUpper.test(password) && password.length >= 8;
```

ตัวอย่าง: ไฮไลต์คำค้นหาในผลลัพธ์

หน้าค้นหาสินค้ามักไฮไลต์คำที่ผู้ใช้พิมพ์ในผลลัพธ์ — ใช้ `replace()` ร่วมกับ regex แบบ global (g) และ ไม่สนตัวพิมพ์ (i) เพื่อครอบคำนั้นด้วยแท็ก `<mark>`

```
function highlightMatch(text, keyword) {  
  const pattern = new RegExp(keyword, "gi");  
  return text.replace(pattern, (match) => `${match}</mark>`);  
}
```

'เสื้อยืดคอกกลม คือ เสื้อที่ใส่สบาย'

```
highlightMatch("เสื้อยืดคอกกลม คือ เสื้อที่ใส่สบาย", "เสื้อ");
```

```
// "<mark>เสื้อ</mark>ยืดคอกกลม"
```

```
// ผลลัพธ์นี้จะนำไปแสดงในหน้าเว็บ คำว่า 'เสื้อ' จะถูก
```

```
// ครอบด้วย <mark> ทำให้ browser เน้นสีพื้นหลังให้อัตโนมัติ
```

08

EXCEPTION HANDLING

จัดการข้อผิดพลาดอย่างเป็นระบบ

try/catch/finally: จับข้อผิดพลาดไม่ให้แอปพลิเคชันพัง

- try — ใส่โค้ดที่อาจเกิด error ไว้ในนี้
- catch (error) — ทำงานเมื่อเกิด error ขึ้นใน try เท่านั้น
- finally — ทำงานเสมอ ไม่ว่าจะเกิด error หรือไม่ (เช่น ปิด loading state)
- ถ้าไม่จับ error ไว้ โปรแกรมทั้งหน้าจะหยุดทำงานทันที
- สามารถเขียนแค่ try { ... } finally { ... } โดยไม่มี catch ได้

```
try {  
  const data = JSON.parse(invalidJson);  
  console.log(data);  
} catch (error) {  
  console.error("แปลงข้อมูลไม่สำเร็จ:", error.message);  
} finally {  
  setLoading(false);  
}
```

throw สร้าง Error ของเราเองได้

- throw หยุดการทำงานทันที และส่งค่าไปให้ catch ที่ใกล้เคียงที่สุดจับ
- new Error (message) สร้าง object error มาตรฐาน มี .message และ .name
- โยน error ตั้งแต่เนิ่น ๆ เมื่อพบข้อมูลผิดปกติ ดีกว่าปล่อยให้พังภายหลัง

```
function calculateShippingFee(total) {  
  if (total < 0) {  
    throw new Error("ยอดสั่งซื้อต้องไม่ติดลบ");  
  }  
  return total > 1000 ? 0 : 50;  
}  
  
try {  
  calculateShippingFee(-100);  
} catch (error) {  
  console.log(error.message);  
  // "ยอดสั่งซื้อต้องไม่ติดลบ"  
}
```

สร้าง Error class ของตัวเอง (extends Error)

- extends Error สืบทอดคุณสมบัติพื้นฐาน (.message, .stack) มาได้
- ตั้งชื่อ .name เฉพาะของตัวเอง เพื่อแยกประเภท error ได้ง่าย
- เพิ่ม property พิเศษได้ เช่น รหัสสถานะ (statusCode) จาก API
- ใน catch เช็คด้วย instanceof เพื่อจัดการ error แต่ละประเภทต่างกัน

```
class ApiError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name = "ApiError";
    this.statusCode = statusCode;
  }
}

try {
  throw new ApiError("ไม่พบสินค้า", 404);
} catch (error) {
  if (error instanceof ApiError) {
    console.log(error.statusCode, error.message);
  }
}
```


09

JAVASCRIPT MODULE

แยกโค้ดเป็นไฟล์ย่อยด้วย import/export

Module คืออะไร ทำไมพัฒนาโปรเจกต์ต้องใช้

- แต่ละไฟล์ .js คือ 1 module มีขอบเขตตัวแปรของตัวเอง (ไม่ปนกับไฟล์อื่น)
- แยกโค้ดตามหน้าที่ เช่น api.js, utils.js, components/Button.js
- นำฟังก์ชัน/ค่าที่ต้องใช้ร่วมกัน 'ส่งออก' (export) แล้ว 'นำเข้า' (import) ไปใช้ในไฟล์อื่น
- ช่วยให้โค้ดดูแลง่าย ทดสอบง่าย และนำกลับมาใช้ซ้ำได้ (reusability)

```
// โครงสร้างไฟล์ทั่วไปของโปรเจกต์ frontend
src/
├── utils/
│   ├── formatPrice.js
│   └── validateForm.js
├── api/
│   └── productApi.js
└── main.js

// main.js นำเข้าฟังก์ชันจากไฟล์อื่นมาใช้
import { formatPrice } from "./utils/formatPrice.js";
```

Common JS

- ใส่ export หน้าตัวแปร/ฟังก์ชันที่ต้องการส่งออก ได้หลายตัวในไฟล์เดียว
- ตอน import ต้องใช้ 'ชื่อเดียวกัน' ใน { } กับตอน export เสมอ
- เลือก import เฉพาะตัวที่ต้องใช้ได้ ไม่ต้องดึงมาทั้งหมด

```
// greeting.js
function greeting(name) {
  return `hello ${name}`;
}

const greetWords = ['hello', 'hi', 'hey'];
module.exports = { greeting, greetWords }

// main.js
const { greeting, greetWords } = require('./greeting.js')
console.log(greeting('Mary')) //hello Mary
console.log(greetWords) //[ 'hello', 'hi', 'hey' ]
```

ES Module: Named Export: ส่งออกได้หลายค่าในไฟล์เดียว

- ใส่ export หน้าตัวแปร/ฟังก์ชันที่ต้องการส่งออก ได้หลายตัวในไฟล์เดียว
- ตอน import ต้องใช้ 'ชื่อเดียวกัน' ใน { } กับตอน export เสมอ
- เลือก import เฉพาะตัวที่ต้องใช้ได้ ไม่ต้องดึงมาทั้งหมด

```
// utils/formatPrice.js
export function formatPrice(n) {
    return n.toLocaleString() + " บาท";
}

export const TAX_RATE = 0.07;

// main.js
import { formatPrice, TAX_RATE } from "./utils/formatPrice.js";

console.log(formatPrice(1500)); // "1,500 บาท"
```

ES Module: Default Export: ส่งออกได้แค่ 1 ค่าต่อไฟล์

- แต่ละไฟล์มี default export ได้แค่ 1 ค่าเท่านั้น (ต่างจาก named ที่มีได้หลายค่า)
- ตอน import ตั้งชื่อเป็นอะไรก็ได้ ไม่ต้องตรงกับตอน export (ไม่ต้องใช้ { })
- นิยมใช้กับไฟล์ที่ส่งออกค่าหลักของไฟล์นั้น เช่น component หรือ class เดียว
- 1 ไฟล์สามารถมีทั้ง default และ named export ผสมกันได้

```
// components/Button.js
export default function Button({ label }) {
  return `<button>${label}</button>`;
}

// main.js - ตั้งชื่อตอน import อะไรก็ได้
import MyButton from "./components/Button.js";

console.log(MyButton({ label: "ส่งชื่อ" }));
```

ES Module: รูปแบบ Import ที่ใช้บ่อยในโปรเจกต์จริง

```
// 1) ตั้งชื่อใหม่ตอน import (as)
import { formatPrice as toBaht } from "./utils/formatPrice.js";

// 2) นำเข้าทั้งหมดเป็นก้อนเดียว (namespace import)
import * as priceUtils from "./utils/formatPrice.js";
priceUtils.formatPrice(500);

// 3) ผสม default + named import ในบรรทัดเดียว
import Button, { ButtonGroup } from "./components/Button.js";
```

ตัวอย่าง: แยกโค้ด API ออกจากหน้าหลัก

แยกฟังก์ชันเรียก API ไว้คนละไฟล์กับหน้าที่แสดงผล ทำให้ทดสอบและแก้ไข endpoint ได้โดยไม่กระทบโค้ดส่วนแสดงผล

```
// api/productApi.js
export async function getProducts() {
  const res = await fetch("/api/products");
  return res.json();
}

// pages/ShopPage.js
import { getProducts } from "../api/productApi.js";

const products = await getProducts();
renderProductList(products);
```

แนวทางการใช้ AI ในชั่วโมงปฏิบัติ

1

ลองเขียนเองก่อนเสมอ

ฝึกคิด logic ด้วยตนเองก่อนถาม AI

2

ใช้ AI เพื่อตรวจสอบ ไม่ใช่เพื่อคัดลอก

ถามให้ AI อธิบายว่าโค้ดของตนผิดตรงไหน แทนที่จะขอโค้ดเฉลยทั้งหมด

(ให้ใช้ AI ถามได้เพียง 1 ครั้ง ต่อ โจทย์ และห้ามถามเพื่อหา solution) ให้ระบุครั้งที่ถาม และ prompt ที่ใช้

3

อธิบายโค้ดได้เสมอ

หากใช้โค้ดจาก AI ต้องอธิบายการทำงานที่ละบรรทัดได้เมื่อถูกถาม

สรุปบทเรียน

ทบทวน - ลงมือทำ Lab Sheet

เปิด Lab Sheet แล้วลองทำด้วยตนเอง — การลงมือเขียนโค้ดจริงคือทางลัดที่เร็วที่สุดสู่ความเข้าใจ



คำถาม & แลกเปลี่ยน

Q & A